1.什么是反射

- Reflection(反射) 是 Java 程序开发语言的特征之一，它允许运行中的 Java 程序对自身进行检查，比如：
  - 照镜子，孙悟空照镜子发现自己是个猴，猪八戒照镜子发现自己是个猪
  - 凶手作案是按照事件发展顺序执行，警察需要根据线索推断出凶手是谁
  - 学生按照自己的方式完成作业，老师需要根据作业内容推断出作业是以什么方式完成的
- 反射能干什么
  - 反射可以在运行时获取一个类的所有信息，包括成员变量，成员方法，构造器等
    - 比如：刑侦剧中我们会看到这样的场景，给一个姓名或一张照片，然后要这个人的全部资料
    - 给一个车牌号，要这辆车的所有信息；给一个手机号，要这个号码的全部数据
  - 并且可以操纵类的字段、方法、构造器等部分
  - 被 private 封装的资源只能类内部访问，外部是不行的，但反射能直接操作类私有属性
- 怎么实现反射
  - 反射就是把 java 类中的各种成分映射成一个个的 Java 对象
  - 一个类的成员变量、方法、构造方法、包等等信息，在加入类中都有一个类来描述
  - 比如：脸是你自己的，但是你想看到必须借助镜子；身体是你自己的，但你想知道有没有疾病必须借助医疗仪器
  - 姓名，照片，车牌号都是公开的，但是想要获取详细信息必须借有相关部门的数据库做支撑
  - 所以一个类想要获取这个类中的所有信息，必须要借助一个额外的工具类，这个类就是 Class 类
- Class类
  - Class 类的实例表示正在运行的 Java 应用程序中的类和接口
  - 也就是 jvm 中有 N 多的实例，每个类都有该Class对象（包括基本数据类型）
  - Class 对象的由来是将 .class 文件读入内存，并为之创建一个 Class 对象
  - 在加载类时由 JVM 以及通过调用类加载器中的 defineClass 方法自动构造的
  - 也就是这不需要我们自己去处理创建，JVM 已经帮我们创建好了

2.反射API

2.1 获取Class对象

- Class 是反射机制的入口，必须有 Class 才能进行后续操作
- Class 表示加载到 JVM 中的那个类，被称为类实例
- Class 中存储了关于类中的所有元数据
- 在运行期间，一个类，只有一个 Class 对象产生

2.1.1 关于实例化对象

- 直接用此对象调用 getClass 方法

```
Person p = new Person();
Class clazz = p.getClass();
```

- 注意
  - 此处使用的是 Object 类中的 getClass() 方法，因为所有类都继承 Object 类

- native 方法是 Java 调用非 Java 代码的接口，由非 Java 语言实现，比如用 C 或 C++ 实现
- 定义一个 native 方法时，并不提供实现体（比较像定义一个Java Interface），因为其实现体是由非 Java 语言在外面实现的

2.1.2 关于类

- 调用类的 class 属性

```
Person p = new Person();
Class clazz = Person.class;
```

2.1.3 关于类路径

- 调用 Class 类中的静态方法 forName()

```
Class.forName("com.mysql.cj.jdbc.Driver")
```

2.2 常用方法

- 父类准备

```
package oracle;

public class Biology {

    //父类公开属性
    public double height;
    //父类私有属性
    private double weight;

    //父类公开构造方法
    public Biology() {
    }

    //父类私有构造方法
    private Biology(double height, double weight) {
        this.height = height;
        this.weight = weight;
    }

    //父类公开构造方法
    public Biology(double height) {
        this.height = height;
    }

    //父类公开普通方法
    public void speak(){
        System.out.println("生物可以交流");
    }

    //父类私有普通方法
    private String eatFood(String food){
        System.out.println("生物需要吃东西");
        return food;
    }
```

```
}
```

- 子类准备

```java
package oracle;

public class Person extends Biology{

    //私有属性
    private int id;
    private String name;
    //公开属性
    public String gender;

    //公开构造方法
    public Person() {
        super();
    }

    //私有构造方法
    private Person(int id, String name) {
        this.id = id;
        this.name = name;
    }

    //公开普通方法
    public void walk(){
        System.out.println("我是人类，可以直立行走");
    }

    //私有普通方法
    private String jump(int height){
        return "我是人类，可以跳跃" + height + "米";
    }
}
```

2.2.1 关于包名类名和类

- 关于包名类名和类相关的方法

```java
Person p = new Person();
Class clazz = p.getClass();

Biology biology = new Biology();
Class aClass = biology.getClass();

//获取当前类所在包名
//结果  oracle
String packageName = clazz.getPackage().getName();
System.out.println(packageName);

//获取类名
//结果  Person
String simpleName = clazz.getSimpleName();
System.out.println(simpleName);

```

```java
//获取完整类名
//结果  oracle.Person
String allName = clazz.getName();
System.out.println(allName);

//通过反射创建实例,此方法在 JDK9 版本开始，因为其对异常处理的不是很好且不能调用有参构造方法，而变为过期方法
Person pClazz = (Person) clazz.newInstance();

//获取当前Class对象的访问修饰符,每种访问修饰符都有相应的数值与之对应,都是二进制表示的数
int m = clazz.getModifiers();
//可以输出每种修饰符对应的数字
System.out.println(Modifier.PUBLIC)
//把数字转成具体有哪些修饰符
System.out.println(Modifier.toString(m));
//判断是否存在某个修饰符
System.out.println(Modifier.isFinal(m));

//获取当前类的父类
Class superclass = clazz.getSuperclass();
System.out.println(superclass);

//获取当前类的所有内部类
Class[] classes = clazz.getClasses();
System.out.println(Arrays.toString(classes));

//判断参数类是否为当前反射类的子类及多重子类
boolean assignableFrom = aClass.isAssignableFrom(Person.class);
System.out.println(assignableFrom);

//父类 Biology<T1,T2>   子类 Person extends Biology<String,Integer>
//如果父类是参数化类型，可以获取父类的泛型 返回值是Type，选择使用参数化类型子类接收
ParameterizedType genericSuperclass = (ParameterizedType)arrayListClass.getGenericSuperclass();
//把参数化类型转成我们可以读取的类型参数的形式
Type[] actualTypeArguments = genericSuperclass.getActualTypeArguments();
System.out.println(Arrays.toString(actualTypeArguments));
```

2.2.2 关于属性

- 关于属性的方法

```java
Person p = new Person();
Class clazz = p.getClass();

/*获取所有公开的属性，包括父类属性
 *结果  [public java.lang.String oracle.Person.gender,
 *      public double oracle.Biology.height]
 */
Field[] fields = clazz.getFields();
System.out.println(Arrays.toString(fields));

/*获取所有公开和私有的属性,但不包括父类的属性
 *结果  [private int oracle.Person.id,
 *      private java.lang.String oracle.Person.name,
 *      public java.lang.String oracle.Person.gender]
```

```
 */
Field[] declaredFields = clazz.getDeclaredFields();
System.out.println(Arrays.toString(declaredFields));

//获取指定公开的属性信息，包括继承变量
//结果  public double oracle.Biology.gender
Field gender = clazz.getField("gender");
System.out.println(gender);

//获取指定公开或私有的属性,但不包括继承的变量
//结果  private java.lang.String oracle.Person.name
Field name = clazz.getDeclaredField("name");
System.out.println(name);

//获取当前字段名字
System.out.println(name.getName());
//获取当前字段修饰符
System.out.println(name.getModifiers());
//获取当前字段的类型
System.out.println(name.getType());


Person p = (Person)clazz.newInstance();
//允许访问当前的私有变量，如果不设置，私有变量依然用不了
name.setAccessible(true);
//从p对象中获取此字段的值 类似于被动语态
System.out.println(name.get(p));
//对p对象中的此字段赋值
name.set(p,"haha");
System.out.println(name.get(p));
```

- 案例：写一个 insert(Object obj) 通用的添加方法

```
/**
 * 可以对任意类型所对应的表进行插入数据
 * 要求：类名必须与表名相同，属性名必须与表中字段同名
 * @param obj
 */
public boolean insertObject(Object obj){

    boolean flag = true;
    try{

        //获取当前对象的Class对象
        Class clazz = obj.getClass();

        //获取当前类的所有的属性，用于添加表数据时，指定字段名
        Field[] fields =  clazz.getDeclaredFields();

        StringBuffer sb = new StringBuffer();

        sb.append("insert into " + clazz.getSimpleName().toLowerCase()+"
("");

        //通过遍历类中所有的属性，添加了sql语句中需要添加的字段名
        for (int i = 0; i < fields.length; i++) {
            sb.append(fields[i].getName().toLowerCase());
```

```
            sb.append((i==fields.length-1)?"":",");
        }

        sb.append(") values (");

        //有多少个属性，就有多少个占位符
        for (int i = 0; i < fields.length; i++) {
            sb.append((i== fields.length-1?"?)":"?,"));
        }

        System.out.println(sb);

        conn = ConnectionUtils.getConnection();
        ps = conn.prepareStatement(sb.toString());

        //对所有的占位符进行赋值
        for (int i = 0; i < fields.length; i++) {
            fields[i].setAccessible(true);
            ps.setObject(i+1,fields[i].get(obj));
        }

        int result = ps.executeUpdate();

        if(result!=1){
            flag = false;
        }
    }catch (Exception e){
        e.printStackTrace();
    }finally {
        ConnectionUtils.closed(conn,ps);
    }
    return flag;
}
```

2.2.3 关于构造方法

- 关于构造方法的反射方法

```
Person p = new Person();
Class clazz = p.getClass();

//获取当前类所有公开构造方法
//结果  [public oracle.Person()]
Constructor[] constructors = clazz.getConstructors();
System.out.println(Arrays.toString(constructors));

/**
 * 获取当前类所有构造方法
 * 结果  [public oracle.Person(),
 *       private oracle.Person(int,java.lang.String)]
 */
Constructor[] declaredConstructors = clazz.getDeclaredConstructors();
System.out.println(Arrays.toString(declaredConstructors));

//获取当前类指定的公开构造方法
//结果 public oracle.Person()
Constructor constructor = clazz.getConstructor();
```

```
System.out.println(constructor);
declaredConstructor.newInstance();

//获取当前类指定的构造方法
//结果  private oracle.Person(int,java.lang.String)
Constructor declaredConstructor =
    clazz.getDeclaredConstructor(int.class, String.class);
System.out.println(declaredConstructor);
declaredConstructor.newInstance(1,"tom");
```

2.2.4 关于普通方法

- 关于普通方法的反射方法

```
Person p = new Person();
Class clazz = p.getClass();

/**
 *获取当前类所有公开普通方法，包括继承的方法
 *结果  [public void oracle.Person.walk(),
 *      public void oracle.Biology.speak(),
 *      public final void java.lang.Object.wait()
 *                      throws java.lang.InterruptedException,
 *      public final void java.lang.Object.wait(long,int)
 *                      throws java.lang.InterruptedException,
 *      public final native void java.lang.Object.wait(long)
 *                      throws java.lang.InterruptedException,
 *      public boolean java.lang.Object.equals(java.lang.Object),
 *      public java.lang.String java.lang.Object.toString(),
 *      public native int java.lang.Object.hashCode(),
 *      public final native java.lang.Class java.lang.Object.getClass(),
 *      public final native void java.lang.Object.notify(),
 *      public final native void java.lang.Object.notifyAll()]
 */
Method[] methods = clazz.getMethods();
System.out.println(Arrays.toString(methods));

/**
 * 获取当前类所有普通方法，不包括继承的方法
 * 结果  [private java.lang.String oracle.Person.jump(int),
 *       public void oracle.Person.walk()]
 */
Method[] declaredMethods = clazz.getDeclaredMethods();
System.out.println(Arrays.toString(declaredMethods));

//根据方法名及参数类型来获得某一个方法
Method method = clazz.getMethod(String name,Class... parameterTypes);
System.out.println(method);

//获取当前类指定的公开普通方法。包括父类的方法
//结果 public void oracle.Person.walk()
Method walk = clazz.getMethod("walk");
System.out.println(walk);

//获取当前类指定的普通方法，不包括父类的方法
//结果  private java.lang.String oracle.Person.jump(int)
Class[] paraTypes = {int.class};
```

```java
Method declaredMethod = clazz.getDeclaredMethod("jump",paraTypes);
System.out.println(declaredMethod);

//方法的调用，Method被对象调用，相当于被动语态
declaredMethod.setAccessible(true);
System.out.println(jump.invoke(new Person(1, 30));
```